



Vivante Programming:

VGLite[®]

Driver Porting Guide

**for use with
Vivante Vector Graphics
small footprint IP**

Document Revision 1.02

04 March 2022

VERISILICON
LEVEL A: PROPRIETARY INFORMATION

Legal Notices

COPYRIGHT INFORMATION

This document contains proprietary information of Vivante Corporation and VeriSilicon Holdings Co., Ltd. They reserve the right to make changes to any products herein at any time without notice and do not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by Vivante and/or VeriSilicon; nor does the purchase or use of a product from Vivante or VeriSilicon convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of Vivante, VeriSilicon or third parties.

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained in this Vivante proprietary copyright document may be re-distributed by Vivante and VeriSilicon customer SoC vendors who have licensed the related core IP, with the understanding that the material be re-packaged/re-branded as a licensee (SoC vendor) written/labeled document. In the adapted document please identify the technology and features described in this document using the Vivante trademark and include a reference to the copyright owner, for example: "This section describes the Vivante® GPU and contains copyright material disclosed with permission of VeriSilicon®."

(VeriSilicon Distribution **LEVEL A: PROPRIETARY INFORMATION**)

TRADEMARK ACKNOWLEDGMENT

VeriSilicon® and the VeriSilicon logo design are the trademarks or the registered trademarks of VeriSilicon Holdings Co., Ltd. Vivante® is a registered trademark of Vivante Corporation. All other brand and product names may be trademarks of their respective companies.

For our current distributors, sales offices, design resource centers, and product information, visit our web page located at <http://www.verisilicon.com>.

For technical support, please email vivante-support@verisilicon.com.

Vivante and VeriSilicon Proprietary. Copyright © 2022 by Vivante Corporation and VeriSilicon Holdings Co., Ltd. All rights reserved.

Table of Contents

LEGAL NOTICES	2
TABLE OF CONTENTS.....	3
1 OVERVIEW	4
1.1 Requirements/Dependencies	4
1.2 Reference	4
1.2.1 Documents	4
1.2.2 VGLite API and HAL Header Files	4
2 SOFTWARE ARCHITECTURE.....	5
2.1 VGLite Driver Folder Organization	5
2.1.1 User API	5
2.1.2 Kernel Driver.....	5
2.1.3 Kernel HAL Driver	5
3 PORTING THE DRIVER WITH THE VGLITE HAL INTERFACE	7
3.1 Base Address and Contiguous Memory Configuration	7
3.2 Initialization.....	8
3.3 Deinitialization	8
3.4 Allocate/Free Contiguous Video Memory.....	8
3.5 Register Access.....	9
3.6 Map/Unmap Memory	9
3.7 Wait Interrupt	10
3.8 Memory Barrier.....	10
DOCUMENT REVISION HISTORY	11

1 Overview

This document outlines the key tasks required for porting the VGLite driver to a specific OS platform. With the provided reference implementations for Linux, FreeRTOS, and Windows, it is straight forward to port the VGLite driver to other OS platforms. Users just need to implement the HAL porting layer to make VGLite work on a specific OS platform.

VGLite software includes a lightweight API designed for efficient running on Vivante's Vector Graphics (VG) small footprint hardware, including, but not limited to, GCNanoLiteV and GCNanoUltraV hardware.

1.1 Requirements/Dependencies

Linux, FreeRTOS, Windows, or other OS-less environment
VGLite software revision: 3.0.0, releases from November2019 or later

1.2 Reference

1.2.1 Documents

For details on the VGLite API, refer to the document **Vivante VGLite API**, which is included in the VGLite driver release package.

1.2.2 VGLite API and HAL Header Files

For customers with access to VGLite source code:

- The Vivante VGLite API is defined in the file: `<Root_Dir>/inc/vg_lite.h`
- The VGLite HAL interface is defined in the file: `<Root_Dir>/inc/vg_hal_lite.h`

2 Software Architecture

The VGLite API driver is organized into the following components:

- User API functions
- Kernel driver
- Kernel HAL layer

2.1 VGLite Driver Folder Organization

<code><Root_Dir>/inc/...</code>	VGLite header files
<code><Root_Dir>/VGLite/...</code>	VGLite API functions
<code><Root_Dir>/VGLiteKernel/...</code>	VGLite Kernel driver
<code><Root_Dir>/VGLiteKernel/linux/...</code>	VGLite Linux platform-specific HAL implementation
<code><Root_Dir>/VGLiteKernel/rtos/...</code>	VGLite FreeRTOS platform-specific HAL implementation
<code><Root_Dir>/VGLiteKernel/win32/...</code>	VGLite Win32 platform-specific HAL implementation

2.1.1 User API

The User APIs and related enumerations are defined in `<Root_Dir>/inc/vg_lite.h`. The application must include this file for development. The User API functions implement the VGLite APIs as specified in the **VGLite Graphics API** document. The application calls VGLite APIs. The User API functions call the underlying kernel functions to drive the Vivante Vector Graphics hardware.

Refer to subdirectory `<Root_Dir>/VGLite` for User API functions.

2.1.2 Kernel Driver

The VGLite Kernel driver receives calls from User API functions and communicates with Vivante VG hardware to perform VGLite API operations. In the VGLite Linux implementation, the Kernel driver accepts `ioctl` commands from the User API functions to perform different operations. In the FreeRTOS implementation, the "Kernel" driver accepts direct function calls to perform different operations.

NOTE: Kernel resource is shared (include the hardware states), so it should be guaranteed that the kernel module and hardware are initialized only once. Any initialization calls after that should be ignored and counted. The same convention is also applied to destroy. The last destroy call will eventually release the kernel and hardware resource. All "destroy" calls before that are ignored and counted.

Refer to subdirectory `<Root_Dir>/VGLiteKernel` for Kernel Driver functions. For `ioctl` commands, refer to the function `vg_lite_kernel()` in `<Root_Dir>/VGLiteKernel/vg_lite_kernel.c`.

2.1.3 Kernel HAL Driver

The Kernel HAL Layer abstracts some basic hardware operations for the Kernel driver to communicate with Vivante VG hardware. The operations include initialization and de-initialization of hardware, memory allocation and free, register read/write, and CPU/GPU synchronization. The HAL layer is designed to be platform independent so it can be supported on different platforms.

Refer to `<Root_Dir>/inc/vg_lite_hal.h` for HAL layer functions.



3 Porting the Driver with the VGLite HAL Interface

The VGLite driver includes the HAL layer implementations for Linux, FreeRTOS, and Windows. To port the VGLite driver to a new OS platform, users need to implement the Kernel HAL layer to enable VGLite to work on the specific platform.

In some RTOS systems, such as FreeRTOS, there is no concept of User mode and Kernel mode. In these environments the VGLite API functions will simply call the “Kernel” functions in the same memory address space directly.

Check subdirectories `<Root_Dir>/VGLiteKernel/linux`, `<Root_Dir>/VGLiteKernel/rtos`, and `<Root_Dir>/VGLiteKernel/win32` for HAL layer implementations on the different OS platforms.

3.1 Base Address and Contiguous Memory Configuration

```
vg_lite_error_t vg_lite_init_mem(
    uint32_t      register_mem_base,
    uint32_t      gpu_mem_base,
    volatile void * contiguous_mem_base,
    uint32_t      contiguous_mem_size
);
```

This function is used with RTOS systems. It should be called before the initialization is to configure memory-related parameters. The **register_mem_base** specifies the base address for the Vivante VG hardware registers. The **gpu_mem_base** specified the Vivante VG’s memory base address, which is usually set to 0.

Any memory buffer that the Vivante VG hardware operates on must be allocated as a contiguous memory buffer. The **contiguous_mem_base** parameter specifies the reserved contiguous memory for the VGLite driver. The command buffer and tessellation buffer can be allocated from there. The **contiguous_mem_size** specifies the size of the reserved contiguous memory.

3.2 Initialization

```
void vg_lite_hal_initialize(
    void
);
```

This function gets called by the VGLite kernel before the Vivante VG hardware is initialized to allow the SOC to turn on the power or initialize the clocks. The implementation needs to make sure that on exit of this function the power and clock to Vivante Vector Graphics hardware is turned on and stable.

3.3 Deinitialization

```
void vg_lite_hal_deinitialize(
    void
);
```

This function gets called by the VGLite kernel after the Vivante VG hardware is uninitialized by the VGLite kernel, so as to allow the SOC to perform system power control. On exit of this function, it is okay to have the power/clock to the Vivante VG hardware turned off.

3.4 Allocate/Free Contiguous Video Memory

```
vg_lite_error_t vg_lite_hal_allocate_contiguous(
    unsigned long    size,
    void **          logical,
    int32_t *        physical,
    void **          node
);

void vg_lite_hal_free_contiguous(
    void *          memory_handle
);
```

Use this pair of HAL functions to dynamically allocate/free a buffer of contiguous video memory from the platform's memory system. Any memory buffer that the Vivante VG hardware operates on must be allocated as a contiguous memory buffer. The allocated memory buffer will be referenced by the VGLite driver through an opaque handle, usually a pointer to an opaque structure. The platform porting layer can put the necessary information inside this structure.

3.5 Register Access

```
uint32_t vg_lite_hal_peek(
    uint32_t address
);

void vg_lite_hal_poke(
    uint32_t address,
    uint32_t data
);
```

This pair of HAL functions reads data from a Vivante VG register and writes data to a Vivante VG register. The VG register memory space needs to be mapped into a unified address space in the implementation so it can be accessed like a generic memory space.

3.6 Map/Unmap Memory

```
void* vg_lite_hal_map(
    unsigned long size,
    void * logical,
    uint32_t physical,
    uint32_t gpu
);

void vg_lite_hal_unmap(
    void * memory_handle
);
```

Use this pair of HAL functions to map/unmap a contiguous logical or physical system memory buffer into the Vivante VG device memory space. Any system memory buffer, such as a frame buffer or some pre-allocated image or path data, needs to be mapped into the VG device address space and referenced by a memory handle. This allows the VG hardware to access the memory buffer directly. Either a logical or a physical address of a system memory buffer can be passed into the **vg_lite_hal_map** function.

Note: not all OS platforms need to implement these functions. For example, on some RTOS OS, all the memory buffers are statically pre-allocated. Then the map/unmap functions can be empty.

3.7 Wait Interrupt

```
int32_t vg_lite_hal_wait_interrupt(
    uint32_t timeout,
    uint32_t mask,
    uint32_t * value
);
```

This function waits until an interrupt from the Vivante VG hardware has been received. This HAL API is synchronous which means it will not return until the VG hardware generates an interrupt and the interrupt has been received by the operating system.

A timeout value is specified for the function to wait for a specific number of milliseconds for the interrupt to occur. If the interrupt does not occur in the specified timeout period, the function will return with a timeout error.

3.8 Memory Barrier

```
void vg_lite_hal_barrier(
    void
);
```

Some OS systems require a memory barrier to make sure all store operations by the CPU have been handled. This is the wrapper function for a platform specific memory barrier function.

Document Revision History

This section describes top level differences between document revisions.

Document Revision	Date	Compatible SW Release	Notes
1.02	2022-03-04	VGLite SW from November 2021	Filename updated. Legal Notices, General: Update branding layout to include VeriSilicon. Miscellaneous refinements.
1.01	2021-11-19	VGLite SW from November 2021	Filename updated to remove sw version. Compatible hw updated to Vivante VG, including GCNanoUltraV. Section 2.1: Subsection 2.1.1 added for init_mem Miscellaneous refinements
1.00	2019-11-29	from sw3.0.0	Initial version.